

Compte Rendu SAE S1.01 - 02

Initiation au développement

Sommaire

Introduction.....	3
1. Étude théorique des algorithmes.....	3
1.1. Tableau comparatif des complexités.....	3
1.2. Justification des choix algorithmiques.....	4
Les Algorithmes de Tri.....	4
Algorithmes de Recherche et de Filtre.....	5
Algorithme de Répartition (Optimisation).....	5
2. Méthodologie d'évaluation expérimentale.....	6
2.1. Implémentation technique.....	6
2.2. Protocole de test.....	6
3. Comparaison théorique et expérimentale.....	6
3.1. Analyse des algorithmes de filtrage (Linéaire).....	7
3.2. Analyse des algorithmes de tri et d'optimisation (Quadratique).....	8
Conclusion.....	10

Introduction

La nouvelle société Horizons Bus souhaite s'implanter sur le marché du transport en proposant des trajets en bus à travers la France à des tarifs compétitifs. Pour soutenir cette stratégie commerciale et gérer efficacement ses opérations, l'entreprise a besoin d'une application informatique performante capable de traiter les données de ses réservations, actuellement stockées sous forme de fichiers CSV.

Le but de cette SAÉ S1.01-02 est de développer une application modulaire en langage C permettant de gérer ces trajets. Les fonctionnalités attendues vont de l'affichage et la modification des données (ajout/suppression de passagers) à des opérations plus complexes comme le filtrage multicritères et l'optimisation du chiffre d'affaires .

Le présent rapport technique se concentre sur l'analyse algorithmique de la solution développée. Au-delà de la simple réalisation des fonctionnalités attendues, il s'agit d'étudier le comportement du programme face à des volumes de données variables, allant de fichiers tests réduits jusqu'à des bases de 5000 trajets. Ce document présente ainsi l'évaluation expérimentale de la complexité des traitements implémentés, afin de valider les choix techniques et de garantir la performance de l'application lors de sa montée en charge.

1. Étude théorique des algorithmes

1.1. Tableau comparatif des complexités

Pour choisir l'algorithme le plus adapté à notre application (capable de traiter de 10 à 5000 trajets), nous avons comparé les caractéristiques théoriques des principaux tris classiques.

Le tableau ci-dessous résume le principe de fonctionnement et la complexité temporelle (nombre d'opérations élémentaires en fonction de N éléments) pour chaque algorithme.

Algorithme	Principe	Meilleur Cas	Moyen Cas	Pire Cas
Algorithme de Recherche et Filtres	Les fonctionnalités de filtrage reposent sur une stratégie de parcours séquentiel	$O(n)$	$O(n)$	$O(n)$
Tri à Insertion	Insertion de l'élément courant à sa place dans la sous-partie triée.	$O(n)$	$O(n^2)$	$O(n^2)$
Tri Rapide	Partition autour d'un pivot (petits à gauche, grands à droite).	$O(n(\log(n)))$	$O(n(\log(n)))$	$O(n^2)$

1.2. Justification des choix algorithmiques

Cette section explique pourquoi nous avons sélectionné ces algorithmes spécifiques pour répondre aux contraintes de performance (fichiers de 10 à 5000 trajets) et fonctionnelles du projet.

Les Algorithmes de Tri

Nous avons distingué deux cas d'usage nécessitant des performances différentes.

Le Tri par Insertion

Utilisé pour les tris chronologiques et par chiffre d'affaires.

Nous l'avons retenu pour sa stabilité et son efficacité redoutable sur des données "presque triées". Dans le contexte de l'application, l'ajout de quelques nouveaux trajets à une base existante se fait en temps quasi-linéaire $O(n)$, ce qui est plus avantageux qu'un tri complet complexe.

Le Tri Rapide / QuickSort

Utilisé pour trier les passagers par prix lors de l'optimisation.

Ce choix est critique pour la performance. Comme ce tri est appelé à l'intérieur d'une boucle (pour chaque bus déficitaire), nous avons besoin de l'algorithme le plus rapide possible en moyenne $O(n(\log(n)))$ pour éviter d'exploser le temps de calcul global de la fonction d'optimisation.

Algorithmes de Recherche et de Filtre

Pour les filtres multicritères (Ville + Date, Trajets "lendemain"), nous utilisons une Recherche Séquentielle.

Les données n'étant pas indexées sur tous les critères simultanément (le tableau est trié selon un seul critère à la fois), un parcours intégral du tableau $O(n)$ est obligatoire pour garantir que toutes les occurrences correspondant à la demande de l'utilisateur sont trouvées et affichées.

Algorithme de Répartition (Optimisation)

Stratégie :

1. Identification des bus à perte.
2. Tri des passagers par prix décroissant (via QuickSort) pour prioriser les clients à forte valeur.
3. Recherche immédiate du bus de remplacement le plus proche temporellement (choix local optimal).

La structure de l'algorithme implique des boucles imbriquées (parcours des bus multiplié par recherche de remplacement pour chaque passager), ce qui induit une complexité globale tendant vers $O(n^2)$. C'est ce coût quadratique inévitable qui a motivé l'utilisation du QuickSort pour le traitement interne des clients, afin de minimiser la constante de temps.

2. Méthodologie d'évaluation expérimentale

2.1. Implémentation technique

Pour réaliser les mesures de la complexité, nous avons ajouté à notre code une variable globale `compteur_ops` de type `long long` (pour éviter les dépassements de capacité sur les grands fichiers).

Le principe est le suivant :

1. Initialisation : Avant l'appel de la fonction à tester, le compteur est remis à zéro.
2. Instrumentation : À l'intérieur des fonctions (tris, filtres, optimisation), nous incrémentons manuellement ce compteur à chaque opération significative.
 - Comparaison (`if`, `while`) : +1 ou +2 opérations.
 - Affectation (`=`) : +1 opération.
 - Accès mémoire ou calcul arithmétique : +1 opération.
3. Affichage : À la fin du traitement, la valeur du compteur est affichée dans la console.

Nous avons appliqué cette méthode systématiquement sur nos fonctions critiques, notamment `comparerDates`, `trierTrajets` (insertion), `quickSortClients` (tri rapide) et `optimiserReseau`.

2.2. Protocole de test

Les mesures ont été effectuées sur les différents jeux de données fournis, permettant de tracer des courbes d'évolution en fonction de n (nombre de trajets) :

- Fichiers de tests unitaires : 10, 50, 100 trajets
- Fichiers de charge moyenne : 500, 1000 trajets
- Fichier de charge maximale : 5000 trajets

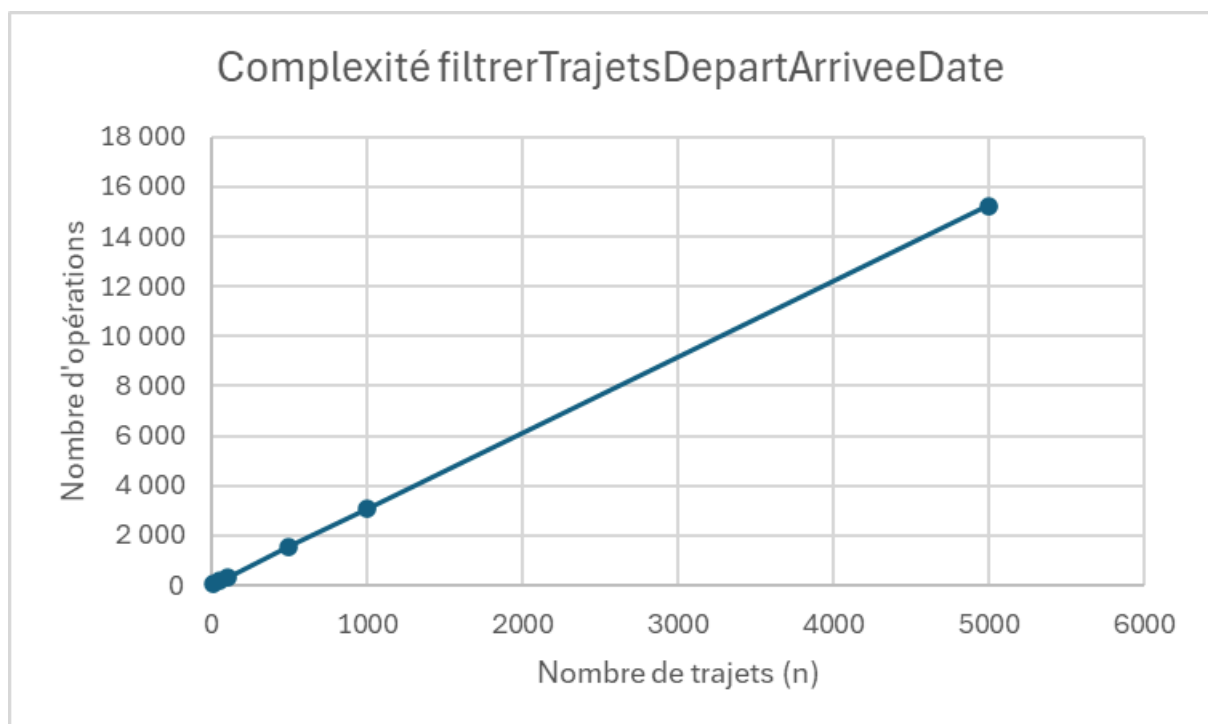
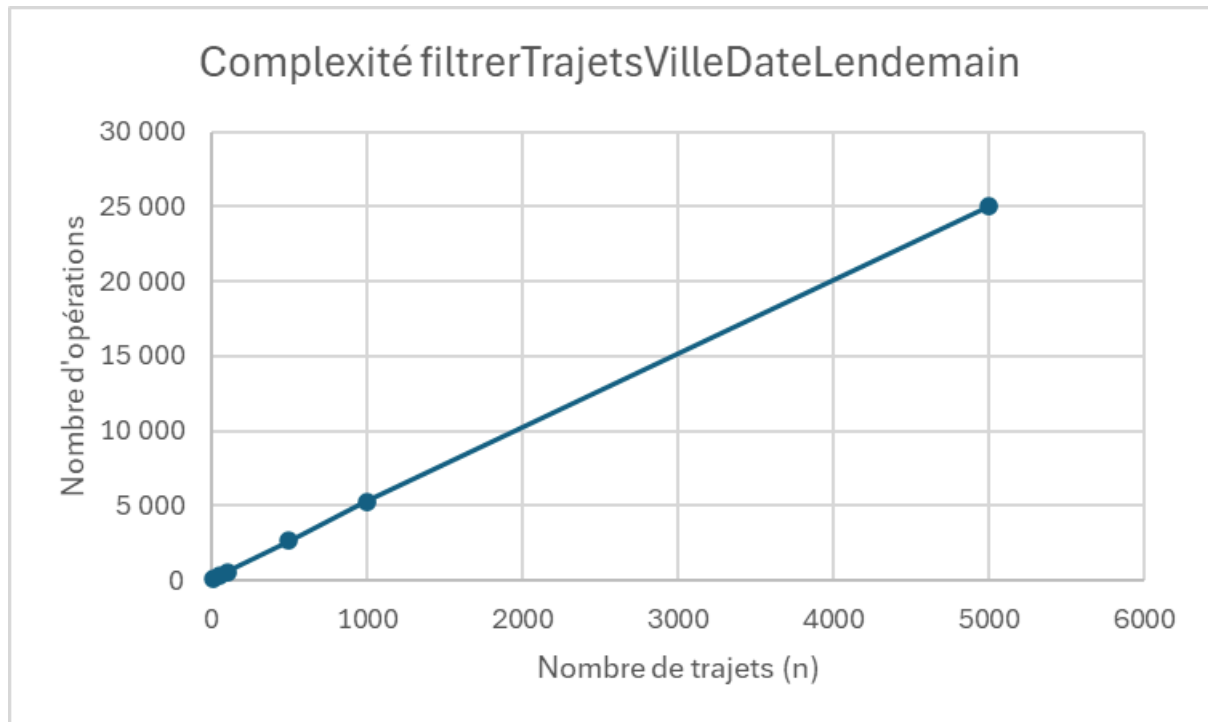
Les résultats obtenus (présentés dans les sections suivantes) correspondent aux valeurs retournées par ce `compteur_ops` lors de l'exécution du programme final.

3. Comparaison théorique et expérimentale

L'analyse des courbes obtenues via nos tests de charge (de 10 à 5000 trajets) nous permet de confronter la complexité théorique à la réalité opérationnelle de l'exécution. Nous avons séparé l'analyse en deux catégories : les algorithmes linéaires et les algorithmes quadratiques.

3.1. Analyse des algorithmes de filtrage (Linéaire)

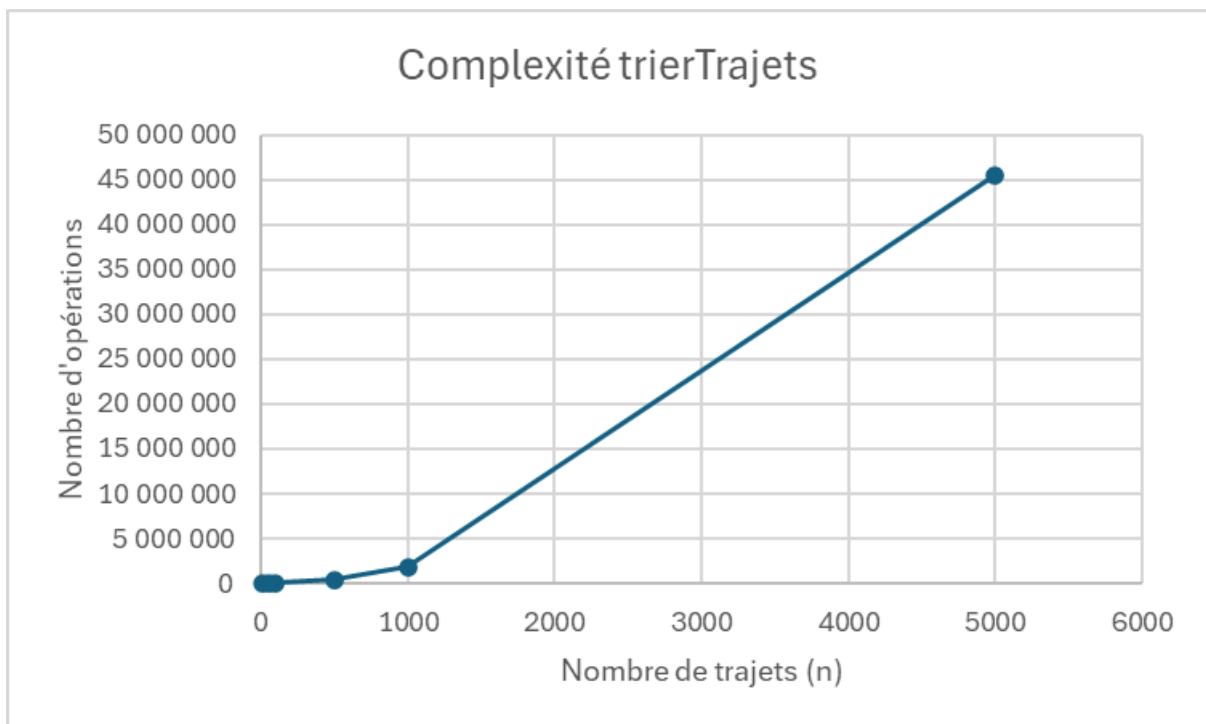
Graphiques analysés : *Complexité filtrerTrajetsDepartArriveeDate* et *Complexité filtrerTrajetsVilleDateLendemain*.

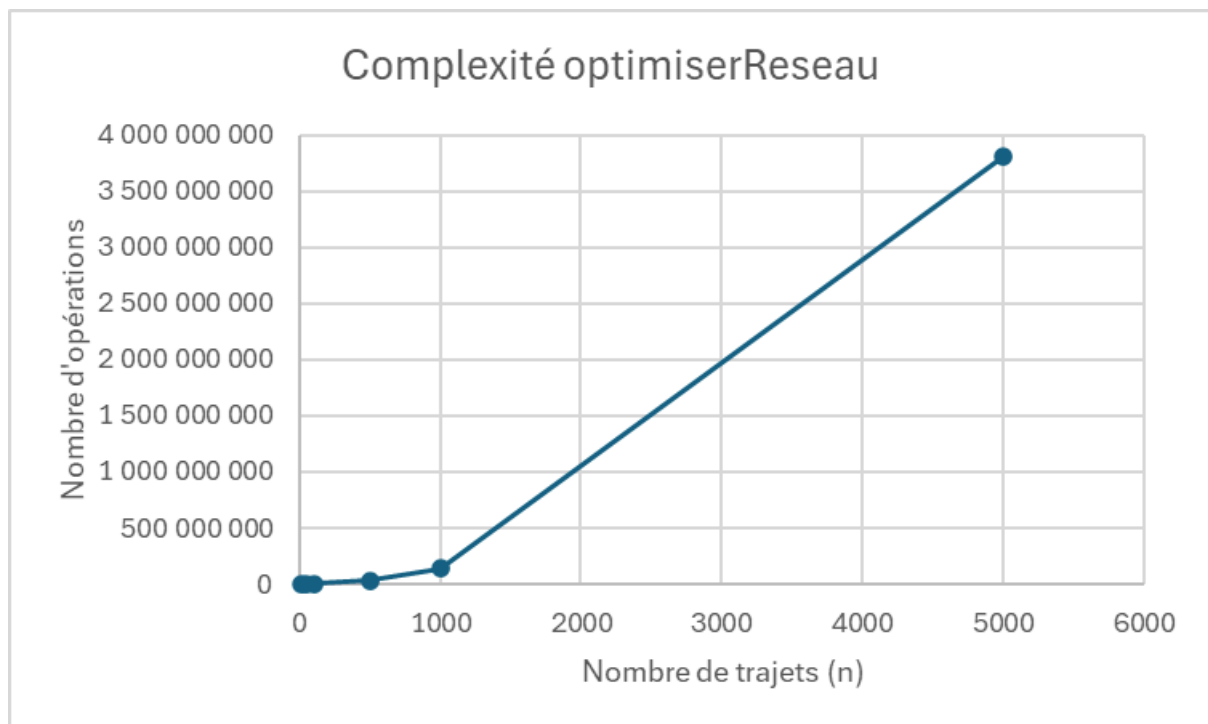
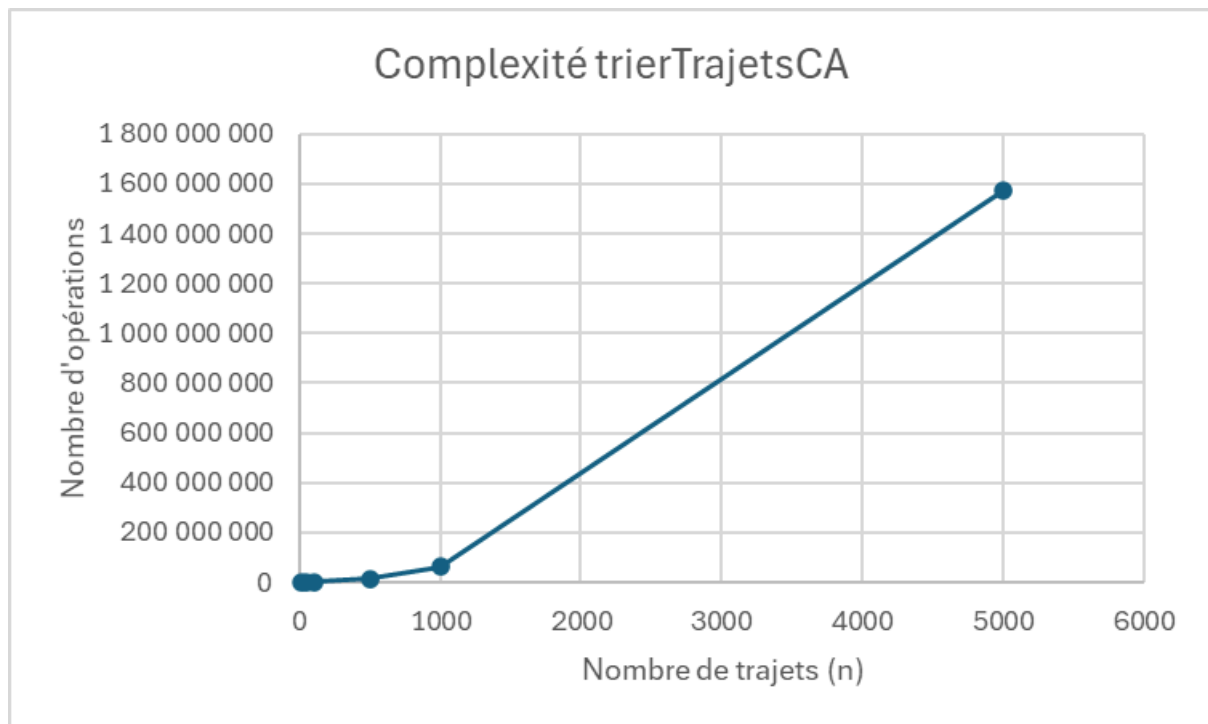


- Ces courbes valident parfaitement la complexité théorique en $O(n)$. La progression du nombre d'opérations est strictement proportionnelle au nombre de trajets.
- Analyse des coefficients :
 - Pour le filtre *TrajetsDepartArriveeDate*, on observe environ 15000 opérations pour 5 000 trajets, soit une moyenne de 3 opérations par trajet. Cela correspond aux comparaisons de chaînes (`strcmp`) et de dates.
 - Pour le filtre *TrajetsVilleDateLendemain*, on relève environ 25000 opérations pour 5000 trajets, soit 5 opérations par trajet. La pente est légèrement plus forte car cet algorithme nécessite des tests arithmétiques supplémentaires sur les horaires.

3.2. Analyse des algorithmes de tri et d'optimisation (Quadratique)

Graphiques analysés : *Complexité trierTrajets*, *Complexité trierTrajetsCA* et *Complexité optimiserReseau*.





- Les trois courbes affichent une allure parabolique caractéristique, où la croissance s'accélère nettement avec l'augmentation de n .
- L'allure des courbes confirme que nous sommes bien dans une classe de complexité $O(n^2)$ (quadratique). Cependant, une disparité immense apparaît sur l'axe des ordonnées (nombre d'opérations) entre ces trois courbes.

Analyse détaillée des écarts de performance :

1. Le tri chronologique (*trierTrajets*)
Pour $n=5000$, nous atteignons approx 45×10^6 opérations. C'est la référence "standard" de notre tri par insertion. Les opérations élémentaires sont simples (comparaisons de dates).
2. Le tri par Chiffre d'Affaires (*trierTrajetsCA*)
Pour $n=5000$, le compteur explose à approx $1,6 \times 10^9$ (1,6 milliard) opérations. Bien que la complexité théorique reste $O(n^2)$, la constante multiplicative est très élevée. La fonction `calculerChiffreAffaires` est appelée à chaque comparaison du tri. Or, cette fonction parcourt les 55 clients du bus. Le coût réel est donc de l'ordre de $O(n^2 \times k)$ où k est le nombre moyen de clients. Visuellement, la courbe est 35 fois plus "haute" que celle du tri chronologique.
3. L'optimisation du réseau (*optimiserReseau*)
Pour $n=5000$, nous atteignons le maximum mesuré de approx. $3,8 \times 10^9$ opérations. Cet algorithme imbrique plusieurs niveaux de boucles, pour chaque bus déficitaire, on parcourt ses clients, et pour chaque client, on parcourt les bus potentiels. Le coût est extrême mais justifié par la complexité du problème (répartition sous contraintes).

L'étude théorique et expérimentale des algorithmes a mis en lumière deux points fondamentaux :

1. Nos mesures confirment que le tri par insertion et l'optimisation suivent une croissance quadratique $O(n^2)$, tandis que les recherches restent linéaires $O(n)$.
2. Nous avons démontré que deux algorithmes de même complexité théorique $O(n^2)$ peuvent avoir des temps d'exécution radicalement différents (d'un facteur 1 à 80) selon le poids des opérations effectuées dans les boucles internes.

Conclusion

Ce projet a permis de réaliser une application fonctionnelle pour Horizons Bus, traitant jusqu'à 5000 trajets conformément au cahier des charges.

Nos mesures valident les modèles théoriques : complexité linéaire $O(n)$ pour les filtres et quadratique $O(n^2)$ pour les tris.

L'analyse souligne surtout l'impact critique des constantes multiplicatives. À complexité égale, le tri par chiffre d'affaires s'avère beaucoup plus coûteux que le tri chronologique en raison des calculs internes, démontrant que l'optimisation des boucles est aussi importante que la classe de complexité.